Assignment Week 4

Gyan Kannur

Problem #1: Math and Extraction with Zero and One-Shot Prompting

```
import os

def _set_env_from_file(var: str, file_path: str = r"C:\Users\gyanr\
gyan-python-workspace\grk_langchain\app\notebooks\data\
openai_key.txt"):
    """
    Reads an API key from a specified file and sets it as an
environment variable.
    """
    if not os.environ.get(var):
        try:
            # The 'with open' statement ensures the file is closed
automatically
            with open(file_path, 'r') as f:
                # Read the first line and strip any leading/trailing
whitespace
                key = f.readline().strip()

            if key:
                os.environ[var] = key
                print(f"Successfully loaded {var} from {file_path}")
            else:
                print(f"Warning: {file_path} is empty.")

        except FileNotFoundError:
            print(f"Error: Key file not found at {file_path}. Please
create the file.")

# --- Execution ---
# Set the environment variable OPENAI_API_KEY from the file


# Verify it was set (optional)

from openai import AsyncOpenAI

open_api_var='OPENAI_API_KEY'
_set_env_from_file(open_api_var)
print(os.environ.get('OPENAI_API_KEY'))
client=AsyncOpenAI()

def generate_inference_request(email_content, prompt):
    # Sanitize
```

```
    email_content_sanitized = email_content.replace("'",
"\\'").replace('"', '\\"')

    # Construct the full prompt with the email content
    sanitized_data = f"{prompt} The email content is as below:
{email_content_sanitized}"

    return sanitized_data
```

Asyncio

I intend to use async api rather than the request api mainly because it is asynchronous. Secondly, this performs better when the response is huge.

```
async def call_async_with_prompt(prompt):
    return await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "user",
                "content": f"{prompt}"
            }
        ],
    )


email_content = """
Hey Jim, I'd like to order some new shoes. Please ship the following:
1. Nike Air Jordan I - 2 pair
2. Converse All-Star - 10 pair
3. New Balance 990 - 1 pair
4. Nike Zoom Fly 5 men's red - 2 pair
Please provide sub-totals and grand total cost.
Thanks.
Lance Gentry
123 Main St.
Chelsea, MI 48109
248-229-2229
"""
```

Zero-shot Prompt

In the activities below prompt engineering approach such as zero-shot, one-shot and using a question and answer format are demonstrated in simplest form to generate better quality json data from an invoice using OpenAI LLM (chatgp-4o-mini).

```
zero_shot_prompt = "Extract all the information from the email as
JSON. Assign a request_type, shoe_price for each shoe, calculate the
```

```
shoe_subtotal for each shoe, and then the grand_total"
data =generate_inference_request(email_content,zero_shot_prompt)
print(data)
```

```
Extract all the information from the email as JSON. Assign a
request_type, shoe_price for each shoe, calculate the shoe_subtotal
for each shoe, and then the grand_total The email content is as below:

Hey Jim, I\'d like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 2 pair
2. Converse All-Star - 10 pair
3. New Balance 990 - 1 pair
4. Nike Zoom Fly 5 men\'s red - 2 pair
Please provide sub-totals and grand total cost.
Thanks.
Lance Gentry
123 Main St.
Chelsea, MI 48109
248-229-2229
```

```
response= await call_async_with_prompt(data)
```

```
response
```

```
ChatCompletion(id='chatcmpl-DRQnYHZUGxOV44vjHZHBR5BebQNzE',
choices=[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='Here’s the extracted
information in JSON format, including the request type, shoe prices,
and calculated subtotals and grand total:\n\n```json\n{\n  "customer":
{\n    "name": "Lance Gentry",\n    "address": "123 Main St., Chelsea,
MI 48109",\n    "phone": "248-229-2229"\n  },\n  "request_type":
"order",\n  "shoes": [\n    {\n      "name": "Nike Air Jordan I",\n
"quantity": 2,\n      "shoe_price": 150.00,\n      "shoe_subtotal":
300.00\n    },\n    {\n      "name": "Converse All-Star",\n
"quantity": 10,\n      "shoe_price": 60.00,\n      "shoe_subtotal":
600.00\n    },\n    {\n      "name": "New Balance 990",\n
"quantity": 1,\n      "shoe_price": 175.00,\n      "shoe_subtotal":
175.00\n    },\n    {\n      "name": "Nike Zoom Fly 5 men\'s red",\n
"quantity": 2,\n      "shoe_price": 120.00,\n      "shoe_subtotal":
240.00\n    }\n  ],\n  "grand_total": 1315.00\n}\n```\n\n**Notes:**\n-
The prices for the shoes used in the JSON are hypothetical. Please
adjust the prices according to your specific requirements if you have
actual prices for the shoes.\n- The grand total is calculated as the
sum of the subtotals for each shoe.', refusal=None, role='assistant',
annotations=[], audio=None, function_call=None, tool_calls=None))],
created=1775430240, model='gpt-4o-mini-2024-07-18',
object='chat.completion', service_tier='default',
system_fingerprint='fp_162d3c480f',
```

```
usage=CompletionUsage(completion_tokens=329, prompt_tokens=153,
total_tokens=482,
completion_tokens_details=CompletionTokensDetails(accepted_prediction_
tokens=0, audio_tokens=0, reasoning_tokens=0,
rejected_prediction_tokens=0),
prompt_tokens_details=PromptTokensDetails(audio_tokens=0,
cached_tokens=0)))
```

Structured Output

Extract content from response object

As seen above the response object provides a lot of information. The required output is embedded with the content field.

Let us use the below to extract the output in json format

```
print(f"Output json:\n {response.choices[0].message.content}")

Output json:
 Here’s the extracted information in JSON format, including the
request type, shoe prices, and calculated subtotals and grand total:

```json
{
  "customer": {
    "name": "Lance Gentry",
    "address": "123 Main St., Chelsea, MI 48109",
    "phone": "248-229-2229"
  },
  "request_type": "order",
  "shoes": [
    {
      "name": "Nike Air Jordan I",
      "quantity": 2,
      "shoe_price": 150.00,
      "shoe_subtotal": 300.00
    },
    {
      "name": "Converse All-Star",
      "quantity": 10,
      "shoe_price": 60.00,
      "shoe_subtotal": 600.00
    },
    {
      "name": "New Balance 990",
      "quantity": 1,
      "shoe_price": 175.00,
      "shoe_subtotal": 175.00
```
```

```
    },
    {
      "name": "Nike Zoom Fly 5 men's red",
      "quantity": 2,
      "shoe_price": 120.00,
      "shoe_subtotal": 240.00
    }
  ],
  "grand_total": 1315.00
}
```

Notes:
- The prices for the shoes used in the JSON are hypothetical. Please adjust the prices according to your specific requirements if you have actual prices for the shoes.
- The grand total is calculated as the sum of the subtotals for each shoe.

Observations

The response of the `zero prompt` is fair. The output json contain **most information** from the email. However it is interesting to notice that the amount attributes such as `shoe_price`,`shoe_subtotal` and `grand_total` are all `arbitrary` even though they were not present in the email text. There is some metadata and `explanation` in the end which is not the format we want. Let us write a function to extract only json from the markdown

Extract json from markup

Below function does few things in order to ensure the required json is extracted

1. Extract the block between `json and`
   Using regex is safer than .find() because it handles variations in whitespace

2. Strip out JavaScript-style comments (// ...) which break json.loads.
3. This specifically targets the "//" and everything following it on that line Usage with your response object:

Extract content data = extract_json_from_markdown(response.choices[0].message.content)

```
import json
import re

def extract_json_from_markdown(content):
    match = re.search(r'```json\s+(.*?)\s+```', content, re.DOTALL)

    if not match:
        raise ValueError("No JSON content found within markdown
tags.")
```

```
    json_string = match.group(1)


    json_string = re.sub(r'//.*', '', json_string)

    # 3. Parse the cleaned string
    try:
        data = json.loads(json_string)
        return data
    except json.JSONDecodeError as e:
        raise ValueError(f"Failed to parse JSON after cleaning: {e}")


json_from_zero_shot =
extract_json_from_markdown(response.choices[0].message.content)
json_from_zero_shot
```

```
{'customer': {'name': 'Lance Gentry',
  'address': '123 Main St., Chelsea, MI 48109',
  'phone': '248-229-2229'},
 'request_type': 'order',
 'shoes': [{'name': 'Nike Air Jordan I',
   'quantity': 2,
   'shoe_price': 150.0,
   'shoe_subtotal': 300.0},
  {'name': 'Converse All-Star',
   'quantity': 10,
   'shoe_price': 60.0,
   'shoe_subtotal': 600.0},
  {'name': 'New Balance 990',
   'quantity': 1,
   'shoe_price': 175.0,
   'shoe_subtotal': 175.0},
  {'name': "Nike Zoom Fly 5 men's red",
   'quantity': 2,
   'shoe_price': 120.0,
   'shoe_subtotal': 240.0}],
 'grand_total': 1315.0}
```

Observations

Now the zero prompt result is as expected in json format but there is scope for improvement.

```
type(zero_shot_prompt)
```

```
str
```

Sanitize input

There are times when there is an extra quote which will fail the function (eg: I'd like to know), hence it is always better to sanitize the input

```
def sanitize_content(inp):
    return inp.replace("'", "\\'").replace('"', '\\"')

email_content_2 = """
Hey Michelle,
I'd like to order some new shoes. Please ship the following:
1. Nike Air Jordan I - 1 pair
2. Converse All-Star - 20 pair
3. New Balance 990 - 2 pair
4. Nike Zoom Fly 5 men's red - 5 pair

Please provide sub-totals and grand total cost.

Thanks.
Artis Gilmore
723 Lexington Blvd.
New York, NY 10001
(503) 484-102
"""
```

One-shot Prompt

In one-shot prompt an example of the task with sample email body and required json output will be provided to the LLM.

```
def create_one_shot_prompt(email_content_2,json_from_zero_shot):

    # Example of the JSON output the model should return
    example_json = f"""
    {json_from_zero_shot}
    """

    # Construct the one-shot prompt
    one_shot_prompt = f"""
    Here is an example of how to extract structured data from an
email:

    Example Email Content:
    {sanitize_content(email_content)}

    Example JSON Output:
    {example_json}

    Now, please extract the same information from the following email
and return it in JSON format:
```

```
    Email Content:
    {email_content_2}
    """

    return one_shot_prompt


# Generate the one-shot prompt
one_shot_prompt =
create_one_shot_prompt(email_content_2,json_from_zero_shot)

print(one_shot_prompt)


    Here is an example of how to extract structured data from an
email:

    Example Email Content:

Hey Jim, I\'d like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 2 pair
2. Converse All-Star - 10 pair
3. New Balance 990 - 1 pair
4. Nike Zoom Fly 5 men\'s red - 2 pair
Please provide sub-totals and grand total cost.
Thanks.
Lance Gentry
123 Main St.
Chelsea, MI 48109
248-229-2229


    Example JSON Output:

    {'customer': {'name': 'Lance Gentry', 'address': '123 Main St.,
Chelsea, MI 48109', 'phone': '248-229-2229'}, 'request_type': 'order',
'shoes': [{'name': 'Nike Air Jordan I', 'quantity': 2, 'shoe_price':
150.0, 'shoe_subtotal': 300.0}, {'name': 'Converse All-Star',
'quantity': 10, 'shoe_price': 60.0, 'shoe_subtotal': 600.0}, {'name':
'New Balance 990', 'quantity': 1, 'shoe_price': 175.0,
'shoe_subtotal': 175.0}, {'name': "Nike Zoom Fly 5 men's red",
'quantity': 2, 'shoe_price': 120.0, 'shoe_subtotal': 240.0}],
'grand_total': 1315.0}


    Now, please extract the same information from the following email
and return it in JSON format:
```

```
    Email Content:

Hey Michelle,
I'd like to order some new shoes. Please ship the following:
1. Nike Air Jordan I - 1 pair
2. Converse All-Star - 20 pair
3. New Balance 990 - 2 pair
4. Nike Zoom Fly 5 men's red - 5 pair

Please provide sub-totals and grand total cost.

Thanks.
Artis Gilmore
723 Lexington Blvd.
New York, NY 10001
(503) 484-102
```

```
one_shot_prompt_response= await
call_async_with_prompt(one_shot_prompt)

json_from_one_shot =
extract_json_from_markdown(one_shot_prompt_response.choices[0].message
.content)
json_from_one_shot
```

```
{'customer': {'name': 'Artis Gilmore',
  'address': '723 Lexington Blvd., New York, NY 10001',
  'phone': '(503) 484-102'},
 'request_type': 'order',
 'shoes': [{'name': 'Nike Air Jordan I',
   'quantity': 1,
   'shoe_price': 150.0,
   'shoe_subtotal': 150.0},
  {'name': 'Converse All-Star',
   'quantity': 20,
   'shoe_price': 60.0,
   'shoe_subtotal': 1200.0},
  {'name': 'New Balance 990',
   'quantity': 2,
   'shoe_price': 175.0,
   'shoe_subtotal': 350.0},
  {'name': "Nike Zoom Fly 5 men's red",
   'quantity': 5,
   'shoe_price': 120.0,
   'shoe_subtotal': 600.0}],
 'grand_total': 2300.0}
```

Observations

I have used the zero-shot-prompt response to as an example to create the one shot prompt and the result is of the same format as the json formatted zero shot prompt. Also note that this time the one-shot prompt did not output any `meta-commentary` which helps avoiding to call another function to extract json

```
async def call_async_with_prompt(prompt, client):
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "user", "content": prompt}
        ],
    )
    return response
```

Few shot Prompt

I am now going to use a question and answer format prompting to steer the llm to provide the answers in the format required. Although this may not be a chain of thought explicitly, let us see how it behaves

```
def create_q_a_prompt(que1,ans1,que2):

   full_prompt= f"""
    Input:
    {sanitize_content(que1)}

    Output:
    {ans1}

    Input:
    {sanitize_content(que2)}

    Output:
    """
   return full_prompt

expected_output={
    "shoe_agent_name": "Jim",
    "request_type": "Order",
    "customer_name": "Lance Gentry",
    "customer_street": "123 Main St.",
    "customer_city": "Chelsea",
    "customer_state": "MI",
    "customer_zip": 48109,
    "customer_phone": "248-229-2229",

    "items": [
        {
```

```
            "shoe_brand": "Nike",
            "shoe_model": "Air Jordan I",
            "shoe_quantity": 2,
            "shoe_price": 100,
            "shoe_subtotal": 200
        },
        {
            "shoe_brand": "Converse",
            "shoe_model": "All-Star",
            "shoe_quantity": 10,
            "shoe_price": 30,
            "shoe_subtotal": 300
        },
        {
            "shoe_brand": "New Balance",
            "shoe_model": "990",
            "shoe_quantity": 1,
            "shoe_price": 110,
            "shoe_subtotal": 110
        },
        {
            "shoe_brand": "Nike",
            "shoe_model": "Zoom Fly 5 men's red",
            "shoe_quantity": 2,
            "shoe_price": 400,
            "shoe_subtotal": 800
        }
    ],
    "grand_total": 1410
}
```


```
multiple_q_a =
create_q_a_prompt(email_content,str(expected_output),email_content_2)
```

```
multiple_q_a
```


```
'\n    Input:\n    \nHey Jim, I\\\'d like to order some new shoes.
Please ship the following: \n1. Nike Air Jordan I - 2 pair \n2.
Converse All-Star - 10 pair \n3. New Balance 990 - 1 pair \n4. Nike
Zoom Fly 5 men\\\'s red - 2 pair\nPlease provide sub-totals and grand
total cost.\nThanks.\nLance Gentry\n123 Main St.\nChelsea, MI 48109\
n248-229-2229\n\n\n    Output:\n
{\'shoe_agent_name\': \'Jim\', \'request_type\': \'Order\', \'customer
_name\': \'Lance Gentry\', \'customer_street\': \'123 Main
St.\', \'customer_city\': \'Chelsea\', \'customer_state\': \'MI\', \'c
ustomer_zip\': 48109, \'customer_phone\': \'248-229-2229\', \'items\':
[{\'shoe_brand\': \'Nike\', \'shoe_model\': \'Air Jordan
I\', \'shoe_quantity\': 2, \'shoe_price\': 100, \'shoe_subtotal\':
200}, {\'shoe_brand\': \'Converse\', \'shoe_model\': \'All-
Star\', \'shoe_quantity\': 10, \'shoe_price\': 30, \'shoe_subtotal\':
```

```
300}, {\'shoe_brand\': \'New
Balance\', \'shoe_model\': \'990\', \'shoe_quantity\':
1, \'shoe_price\': 110, \'shoe_subtotal\': 110},
{\'shoe_brand\': \'Nike\', \'shoe_model\': "Zoom Fly 5 men\'s
red", \'shoe_quantity\': 2, \'shoe_price\': 400, \'shoe_subtotal\':
800}], \'grand_total\': 1410}\n\n    Input:\n    \nHey Michelle,\
nI\\\'d like to order some new shoes. Please ship the following:\n1.
Nike Air Jordan I - 1 pair\n2. Converse All-Star - 20 pair\n3. New
Balance 990 - 2 pair\n4. Nike Zoom Fly 5 men\\\'s red - 5 pair\n\
nPlease provide sub-totals and grand total cost.\n\nThanks.\nArtis
Gilmore\n723 Lexington Blvd.\nNew York, NY 10001\n(503) 484-102\n\n\n
Output:\n    '
```

```
combined_result = await call_async_with_prompt(multiple_q_a,client)
```

```
combined_result
```

```
ChatCompletion(id='chatcmpl-DRQujOOKK2ocD1XYM6oZ4te6zjwxb',
choices=[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='```json\n{\n
"shoe_agent_name": "Michelle",\n    "request_type": "Order",\n
"customer_name": "Artis Gilmore",\n    "customer_street": "723
Lexington Blvd.",\n    "customer_city": "New York",\n
"customer_state": "NY",\n    "customer_zip": 10001,\n
"customer_phone": "(503) 484-102",\n    "items": [\n        {\n
"shoe_brand": "Nike",\n            "shoe_model": "Air Jordan I",\n
"shoe_quantity": 1,\n            "shoe_price": 100,\n
"shoe_subtotal": 100\n        },\n        {\n            "shoe_brand":
"Converse",\n            "shoe_model": "All-Star",\n
"shoe_quantity": 20,\n            "shoe_price": 30,\n
"shoe_subtotal": 600\n        },\n        {\n            "shoe_brand":
"New Balance",\n            "shoe_model": "990",\n
"shoe_quantity": 2,\n            "shoe_price": 110,\n
"shoe_subtotal": 220\n        },\n        {\n            "shoe_brand":
"Nike",\n            "shoe_model": "Zoom Fly 5 men\'s red",\n
"shoe_quantity": 5,\n            "shoe_price": 400,\n
"shoe_subtotal": 2000\n        }\n    ],\n    "grand_total": 2910\n}\
n```', refusal=None, role='assistant', annotations=[], audio=None,
function_call=None, tool_calls=None))], created=1775430685,
model='gpt-4o-mini-2024-07-18', object='chat.completion',
service_tier='default', system_fingerprint='fp_c93645df76',
usage=CompletionUsage(completion_tokens=291, prompt_tokens=465,
total_tokens=756,
completion_tokens_details=CompletionTokensDetails(accepted_prediction_
tokens=0, audio_tokens=0, reasoning_tokens=0,
rejected_prediction_tokens=0),
prompt_tokens_details=PromptTokensDetails(audio_tokens=0,
cached_tokens=0)))
```

```
full_res =
extract_json_from_markdown(combined_result.choices[0].message.content)

full_res
```

```
{'shoe_agent_name': 'Michelle',
 'request_type': 'Order',
 'customer_name': 'Artis Gilmore',
 'customer_street': '723 Lexington Blvd.',
 'customer_city': 'New York',
 'customer_state': 'NY',
 'customer_zip': 10001,
 'customer_phone': '(503) 484-102',
 'items': [{'shoe_brand': 'Nike',
   'shoe_model': 'Air Jordan I',
   'shoe_quantity': 1,
   'shoe_price': 100,
   'shoe_subtotal': 100},
  {'shoe_brand': 'Converse',
   'shoe_model': 'All-Star',
   'shoe_quantity': 20,
   'shoe_price': 30,
   'shoe_subtotal': 600},
  {'shoe_brand': 'New Balance',
   'shoe_model': '990',
   'shoe_quantity': 2,
   'shoe_price': 110,
   'shoe_subtotal': 220},
  {'shoe_brand': 'Nike',
   'shoe_model': "Zoom Fly 5 men's red",
   'shoe_quantity': 5,
   'shoe_price': 400,
   'shoe_subtotal': 2000}],
 'grand_total': 2910}
```

Observations

By using a question and answer way of prompting as above, an `example` of `email body` and corresponding expected `json` output were provided in the prompt. The `examples` enabled the LLM to steer it response to the desired output including all the details. The LLM was able to infer the prices of each kind of shoes from the example itself. The output json has all the attributes exactly as mentioned in the example in this case.

Conclusion

There is an alternative method where context is provided to the system before posing the query. Prompt engineering exists at the intersection of art and science; its effectiveness varies significantly depending on the specific model architecture. Because these models are non-deterministic, the same input can yield different results across separate iterations.

Problem #2: Chat Completion for Chain of Thought

```
cot_prompt = """Q: There were nine computers in the server room. Five
more computers were installed each day, from Monday to Thursday. How
many computers are now in the server room?
A: There are 4 days from Monday to Thursday. 5 computers were added
each day. That means in total 4 * 5 = 20 computers were added. There
were 9 computers initially, so now there are 9 + 20 = 29 computers.
The answer is 29.
Q: Michael had 58 golf balls. On Tuesday, he lost 23 golf balls. On
Wednesday, he lost 2 more. How many golf balls did he have at the end
of Wednesday?
A: Michael initially had 58 balls. He lost 23 on Tuesday, so after
that he has 58 - 23 = 35 balls. On Wednesday he lost 2 more so now he
has 35 - 2 = 33 balls. The answer is 33.
Q: Olivia has $23. She bought five bagels for $3 each. How much money
does she have left?
A: She bought 5 bagels for $3 each. This means she spent $15. She has
$8 left.
Q: When I was 6 my sister was half my age. Now I'm 70 how old is my
sister?"""

system_context="You are a logical reasoning assistant. For every
request, follow a systematic Chain of Thought"
```

Using OpenAI's client which uses synchronous invokation

Unlike previous where asyncio was used, here I will use the simple openAI() client and invoke the chat completions method. As seen, there will be the system prompt and the user prompt

```
from openai import OpenAI

client2 = OpenAI()
response4 = client2.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {
            "role": "system",
            "content": f"{system_context}"
        },
        {
            "role": "user",
            "content": f"{cot_prompt}"

        }
    ],
)

print(f"Answer:\n {response4.choices[0].message.content}")
```

```
Answer:
 Let's break down the information step by step.

1. When I was 6, my sister was half my age, which means my sister was
3 years old at that time (6 / 2 = 3).
2. The age difference between me and my sister is 6 - 3 = 3 years.
3. Now I am 70 years old. Since the age difference remains constant,
my sister is 70 - 3 = 67 years old.

Therefore, the answer is that my sister is 67 years old.
```

Observations

`Chain of thought` is a prompting technique that enables reasoning capabilities in LLM responses by providing reasoning examples in the prompt. As observed above, the response yeilded correct result with the use of `chain of thougt` in promt allowing the LLM to understand the mathematical reasoning and generate correct result. It is worth to note that apart from just the answer, LLM response also includes a step by step explanation about how it arrived at the result.